Regularization & Model Selection

ECON 8310: Business Forecasting — Lecture 6

Department of Economics

University of Nebraska at Omaha

October 13, 2026

Lecture Outline

1. The Problem: Too Many Predictors
2. Shrinkage Methods
3. Choosing the Penalty
4. In Practice
5. Key Takeaways and Roadmap

The Problem: Too Many Predictors

Feature engineering gives us more predictors than we can safely estimate. Something has to decide which ones survive.

Why Regularize?

OLS minimizes $\sum_i (y_i - \mathbf{x}_i'\boldsymbol{\beta})^2$. A weekly model with lags, calendar dummies and promotions easily carries $k = 60$ predictors on $n = 150$ weeks: OLS fits that sample beautifully and forecasts badly.

Three things go wrong. The variance of $\hat{\boldsymbol{\beta}}$ grows with k. Correlated predictors get huge coefficients of opposite sign. Past $k > n$ there is no unique solution. **Regularization** adds a penalty on coefficient size:

$$\underbrace{\textstyle\sum_i (y_i - \hat{y}_i)^2}_{\text{fit}} + \underbrace{\lambda \cdot \text{Penalty}(\boldsymbol{\beta})}_{\text{complexity}}$$

You accept a little bias for a large cut in variance. For a forecast, that trade is almost always worth making.

Subset Selection: The Classical Answer

The classical approach *counts* predictors rather than shrinking them (Guyon and Elisseeff 2003).

Best subset fits all 2^k models: exact, and hopeless past $k \approx 30$. **Stepwise** adds or drops one predictor at a time: greedy, so it can miss the best combination. AIC, BIC or CV picks the winner.

> Subset selection is **discrete**: a predictor is in or out, and small changes in the data flip it. The *selected model* is itself high-variance — the instability has moved somewhere harder to see.

FPP §7.5. Shrinkage replaces in-or-out with a continuous dial.

Shrinkage Methods

Keep every predictor, but pull its coefficient toward zero.

Ridge Regression (ℓ_2)

Ridge (Hoerl and Kennard 1970)

$$\hat{\boldsymbol{\beta}}^{\text{ridge}} = \arg\min_{\boldsymbol{\beta}} \sum_{i=1}^{n} (y_i - \mathbf{x}_i'\boldsymbol{\beta})^2 \; + \; \lambda \sum_{j=1}^{k} \beta_j^2$$

Ridge shrinks every coefficient toward zero and sets none exactly to zero: $\lambda = 0$ is OLS, and $\lambda \to \infty$ sends every $\beta_j \to 0$. Two correlated predictors split the effect between them instead of fighting over it.

Use it when most predictors carry a little signal and you want to keep them all.

Standardize first. The penalty is not scale-invariant: the same predictor in dollars and in thousands of dollars is penalized differently, and the units decide what survives.

LASSO (ℓ_1)

LASSO (Tibshirani 1996)

$$\hat{\boldsymbol{\beta}}^{\text{lasso}} = \arg\min_{\boldsymbol{\beta}} \sum_{i=1}^{n} (y_i - \mathbf{x}_i'\boldsymbol{\beta})^2 \; + \; \lambda \sum_{j=1}^{k} |\beta_j|$$

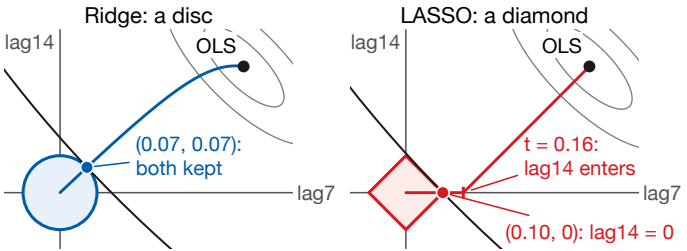
Swap the square for an absolute value and LASSO sets some coefficients **exactly** to zero: selection and estimation in one step, and a sparse model you can read aloud to a stakeholder.

Use it when only a handful of predictors genuinely matter and you want the model to say which.

But be careful what you conclude. Among highly correlated predictors LASSO keeps *one*, more or less arbitrarily. The survivor is not "the important one"; it won a coin toss between near-identical variables.

Why LASSO Zeros Out and Ridge Does Not

Both minimize squared error subject to a budget t: $\sum_j \beta_j^2 \leq t$ or $\sum_j |\beta_j| \leq t$. Ellipses of equal error grow from OLS until they touch the budget. Lab 6's lag7 and lag14, standardized:



two coefficients, constrained form; each λ corresponds to some budget t
colored line: the solution as the budget t grows

The disc has no corners, so ridge keeps both. The diamond's corner zeros lag14, but only while $t < 0.16$, a fifth of OLS's $\sum_j |\hat{\beta}_j| = 0.84$: these lags are nearly interchangeable, so the ellipses run almost along its face. A zero means "not worth its budget here," not "proven irrelevant" (James et al. 2023, §6.2).

Elastic Net: Both Penalties at Once

Elastic Net (Zou and Hastie 2005)

$$\sum_{i=1}^{n}(y_i - \mathbf{x}_i'\boldsymbol{\beta})^2 \; + \; \lambda \left[\alpha \sum_j |\beta_j| + (1-\alpha) \sum_j \beta_j^2 \right]$$

λ sets the total penalty; $\alpha \in [0,1]$ sets the mix ($\alpha = 1$ is LASSO, $\alpha = 0$ is Ridge).

The mix keeps LASSO's ability to zero out predictors while adding Ridge's **grouping effect**: correlated predictors now enter or leave together instead of one surviving arbitrarily.

This is usually the right default for forecasting. Lagged predictors are correlated almost by construction — y_{t-1} and y_{t-2} move together — which is precisely the situation where pure LASSO picks a winner at random and pure Ridge refuses to simplify.

Choosing the Penalty

λ is not estimated from the model. It is tuned — and how you tune it decides whether the model generalizes.

The Regularization Path

Fit across a grid of λ and trace each coefficient. At small λ the coefficients sit near OLS: dense and unstable. As λ grows they shrink, and under LASSO they drop to zero one at a time, until only the intercept — the mean forecast — is left.

The order in which predictors leave is the useful output. The last survivors held up under the most pressure. Report that ordering, not one model's coefficient table: it is far more stable across resamples, and more honest about what the data support.

Choosing λ by Cross-Validation

1. Define a grid of λ, usually log-spaced
2. For each λ, run walk-forward CV and record the mean validation RMSE
3. Pick the λ minimizing CV error, then refit on all training data

The one-standard-error rule: take the *largest* λ whose CV error is within one standard error of the minimum. You give up almost no accuracy for a simpler, more stable model.

> **Never use `KFold` here.** Random folds train on the future and validate on the past: CV looks excellent and the forecast fails (Bergmeir et al. 2018). Use `TimeSeriesSplit`, as in Lecture 1.

In Practice

Standardize, tune on a time-aware split, and read the path.

Regularization in Python

Code Template

```
from sklearn.linear_model import ElasticNetCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import TimeSeriesSplit
model = make_pipeline(StandardScaler(),
ElasticNetCV(l1_ratio=[.1,.5,.7,.9,.95,1],
cv=TimeSeriesSplit(5), random_state=42)).fit(X_tr, y_tr)
```

`make_pipeline` is the part that matters: it keeps standardization *inside* each CV fold. Scale on the full sample and you have leaked the validation set into training, and your CV error is optimistic.

`l1_ratio` is α, the Ridge/LASSO mix; the `...CV` variants tune λ for you.

`sklearn` calls the penalty `alpha`, not `lambda`.

Key Takeaways and Roadmap

Shrinkage is the linear-model counterpart to what trees do by pruning.

Linear Methods: Which One When

Method	Penalty	Zeros out?	Best when
OLS	none	no	$k \ll n$, predictors uncorrelated
Stepwise	discrete	yes	Few candidate predictors, need a story
Ridge	ℓ_2	no	Many predictors, all mildly useful
LASSO	ℓ_1	yes	Few predictors truly matter
Elastic Net	both	yes	Many *correlated* predictors

Connection to Lecture 5. Trees and shrinkage solve the same problem in different geometries. XGBoost's `reg_lambda` is literally a Ridge penalty on leaf weights (`reg_alpha` is the LASSO one), and `gamma` charges a fixed price per leaf, like pruning's α in Lecture 4. Regularization is how every flexible model is kept honest.

Lecture 6: Key Takeaways

1. With many predictors OLS overfits; past $k > n$ it has no unique solution.
2. **Subset selection** is discrete, greedy and unstable.
3. **Shrinkage** is a dial. Ridge shrinks all; LASSO's corners zero some, at small enough budgets; Elastic Net keeps correlated predictors together.
4. **Standardize** inside each CV fold, and **tune** λ with `TimeSeriesSplit`, never random K-fold.
5. The **regularization path**, the order predictors drop out, is more stable than any one coefficient table.

Next: neural networks, where these penalties return as weight decay and dropout.

References I

- Bergmeir, Christoph, Rob J. Hyndman, and Bonsoo Koo (2018). "A Note on the Validity of Cross-Validation for Evaluating Autoregressive Time Series Prediction". In: *Computational Statistics & Data Analysis* 120, pp. 70–83.
- Guyon, Isabelle and André Elisseeff (2003). "An Introduction to Variable and Feature Selection". In: *Journal of Machine Learning Research* 3, pp. 1157–1182.
- Hoerl, Arthur E. and Robert W. Kennard (1970). "Ridge Regression: Biased Estimation for Nonorthogonal Problems". In: *Technometrics* 12.1, pp. 55–67.
- James, Gareth et al. (2023). *An Introduction to Statistical Learning with Applications in Python*. 1st. New York: Springer. URL: https://www.statlearning.com/.
- Tibshirani, Robert (1996). "Regression Shrinkage and Selection via the Lasso". In: *Journal of the Royal Statistical Society: Series B* 58.1, pp. 267–288.
- Zou, Hui and Trevor Hastie (2005). "Regularization and Variable Selection via the Elastic Net". In: *Journal of the Royal Statistical Society: Series B* 67.2, pp. 301–320.